

\$project (aggregation stage)

Definition

`$project` Passes along the documents with the requested fields to the next stage in the pipeline. The specified fields can be existing fields from the input documents or newly computed fields.

Compatibility

`$project` You can use `$project` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$project` stage has the following prototype form:

```
{ $project: { <specification(s)> } }
```

The `$project` takes a document that can specify the inclusion of fields, the suppression of the `_id` field, the addition of new fields, and the resetting of the values of existing fields. Alternatively, you may specify the *exclusion* of fields.

The `$project` specifications have the following forms:

Form	Description
`: <1 or true>`	Specifies the inclusion of a field. Non-zero integers are also treated as `true`.
`_id: <0 or false>`	Specifies the suppression of the `_id` field. To exclude a field conditionally, use the `REMOVE` variable instead. For details, see Exclude Fields Conditionally.
`: `	Adds a new field or resets the value of an existing field. If the expression evaluates to `\$\$REMOVE`, the field is excluded in the output. For details, see Exclude Fields Conditionally.
`: <0 or false>`	Specifies the exclusion of a field. To exclude a field conditionally, use the `REMOVE` variable instead. For details, see Exclude Fields Conditionally. If you specify the exclusion of a field other than `_id`, you cannot employ any other `\$project` specification forms. This restriction does not apply to conditionally exclusion of a field using the `REMOVE` variable. See also the `\$unset` stage to exclude fields.

Behavior

Include Fields

- The `_id` field is, by default, included in the output documents. To include any other fields from the input documents in the output documents, you must explicitly specify the inclusion in `$project`.
- If you specify an inclusion of a field that does not exist in the document, `$project` ignores that field inclusion and does not add the field to the document.

`_id` Field

By default, the `_id` field is included in the output documents. To exclude the `_id` field from the output documents, you must explicitly specify the suppression of the `_id` field in `$project`.

Exclude Fields

If you specify the exclusion of a field or fields, all other fields are returned in the output documents.

```
{ $project: { "<field1>": 0, "<field2>": 0, ... } } // Return all but the speci
```

If you specify the exclusion of a field other than `_id`, you cannot employ any other `$project` specification forms: i.e. if you exclude fields, you cannot also specify the inclusion of fields, reset the value of existing fields, or add new fields. This restriction does not apply to conditional exclusion of a field using the `REMOVE` variable.

See also the `$unset` stage to exclude fields.

Exclude Fields Conditionally

You can use the variable `REMOVE` in aggregation expressions to conditionally suppress a field. For an example, see [Conditionally Exclude Fields](#).

Add New Fields or Reset Existing Fields

MongoDB also provides `$addField` to add new fields to the documents.

To add a new field or to reset the value of an existing field, specify the field name and set its value to some expression. For more information on expressions, see [Expressions](#).

Literal Values

To set a field value directly to a numeric or boolean literal, as opposed to setting the field to an expression that resolves to a literal, use the `$literal` operator. Otherwise, `$project` treats the numeric or boolean literal as a flag for including or excluding the field.

Field Rename

By specifying a new field and setting its value to the field path of an existing field, you can effectively rename a field.

New Array Fields

The `$project` stage supports using the square brackets `[]` to directly create new array fields. If you specify array fields that do not exist in a document, the operation substitutes `null` as the value for that field. For an example, see [Project New Array Fields](#).

You cannot use an array index with the `$project` stage. For more information, see [Array Indexes are Unsupported](#).

Embedded Document Fields

When projecting or adding/resetting a field within an embedded document, you can either use dot notation, as in

```
"contact.address.country": <1 or 0 or expression>
```

Or you can nest the fields:

```
contact: { address: { country: <1 or 0 or expression> } }
```

When nesting the fields, you *cannot* use dot notation inside the embedded document to specify the field, e.g. `contact: { "address.country": <1 or 0 or expression> }` is *invalid*.

Path Collision Errors in Embedded Fields

You cannot specify both an embedded document and a field within that embedded document in the same projection.

The following `$project` stage fails with a `Path collision` error because it attempts to project both the embedded `contact` document and the `contact.address.country` field:

```
{ $project: { contact: 1, "contact.address.country": 1 } }
```

The error occurs regardless of the order in which the parent document and embedded field are specified. The following `$project` fails with the same error:

```
{ $project: { "contact.address.country": 1, contact: 1 } }
```

`$project` Stage Placement

`$project` When you use a `$project` stage it should typically be the last stage in your pipeline, used to specify which fields to return to the client.

Using a `$project` stage at the beginning or middle of a pipeline to reduce the number of fields passed to subsequent pipeline stages is unlikely to improve performance, because the database performs this optimization automatically.

Considerations

Empty Specification

MongoDB returns an error if the `$project` stage is passed an empty document.

For example, running the following pipeline produces an error:

```
db.myCollection.aggregate( [ {  
  $project: { }  
} ] )
```

Array Index

You cannot use an array index with the `$project` stage. For more information, see [Array Indexes are Unsupported](#).

Examples

Include Specific Fields in Output Documents

Consider a `books` collection with the following document:

```
{  
  "_id" : 1,  
  "title" : "abc123",  
  "isbn" : "0001122223334",  
  "author" : { last: "zzz", first: "aaa" },  
  "copies" : 5  
}
```

The following `$project` stage includes only the `_id`, `title`, and the `author` fields in its output documents:

```
db.books.aggregate( [ { $project : { title : 1 , author : 1 } } ] )
```

The operation results in the following document:

```
{ "_id" : 1, "title" : "abc123", "author" : { "last" : "zzz", "first" : "aaa" } }
```

Suppress `_id` Field in the Output Documents

The `_id` field is always included by default. To exclude the `_id` field from the output documents of the `$project` stage, specify the exclusion of the `_id` field by setting it to `0` in the projection document.

Consider a `books` collection with the following document:

```
{
  "_id" : 1,
  "title" : "abc123",
  "isbn" : "0001122223334",
  "author" : { "last" : "zzz", "first" : "aaa" },
  "copies" : 5
}
```

The following `$project` stage excludes the `_id` field but includes the `title`, and the `author` fields in its output documents:

```
db.books.aggregate( [ { $project : { _id: 0, title : 1 , author : 1 } } ] )
```

The operation results in the following document:

```
{ "title" : "abc123", "author" : { "last" : "zzz", "first" : "aaa" } }
```

Exclude Fields from Output Documents

Consider a `books` collection with the following document:

```
{
  "_id" : 1,
  title: "abc123",
  isbn: "0001122223334",
  author: { last: "zzz", first: "aaa" },
  copies: 5,
  lastModified: "2016-07-28"
}
```

The following `$project` stage excludes the `lastModified` field from the output:

```
db.books.aggregate( [ { $project : { "lastModified": 0 } } ] )
```

See also the `$unset` stage to exclude fields.

Exclude Fields from Embedded Documents

Consider a `books` collection with the following document:

```
{
  "_id" : 1,
  title: "abc123",
  isbn: "0001122223334",
  author: { last: "zzz", first: "aaa" },
  copies: 5,
  lastModified: "2016-07-28"
}
```

The following `$project` stage excludes the `author.first` and `lastModified` fields from the output:

```
db.books.aggregate( [ { $project : { "author.first" : 0, "lastModified" : 0 } } ] )
```

Alternatively, you can nest the exclusion specification in a document:

```
db.bookmarks.aggregate( [ { $project: { "author": { "first": 0}, "lastModified"
```

Both specifications result in the same output:

```
{
  "_id" : 1,
  "title" : "abc123",
  "isbn" : "0001122223334",
  "author" : {
    "last" : "zzz"
  },
  "copies" : 5,
}
```

See also the `$unset` stage to exclude fields.

Conditionally Exclude Fields

You can use the variable `REMOVE` in aggregation expressions to conditionally suppress a field.

Consider a `books` collection with the following document:

```
{
  "_id" : 1,
  title: "abc123",
  isbn: "0001122223334",
  author: { last: "zzz", first: "aaa" },
  copies: 5,
  lastModified: "2016-07-28"
}
{
  "_id" : 2,
  title: "Baked Goods",
  isbn: "9999999999999",
  author: { last: "xyz", first: "abc", middle: "" },
  copies: 2,
  lastModified: "2017-07-21"
}
{
  "_id" : 3,
  title: "Ice Cream Cakes",
  isbn: "8888888888888",
  author: { last: "xyz", first: "abc", middle: "mmm" },
  copies: 5,
  lastModified: "2017-07-22"
}
```

The following `$project` stage uses the `REMOVE` variable to excludes the `author.middle` field only if it equals `""`:

```

db.books.aggregate( [
  {
    $project: {
      title: 1,
      "author.first": 1,
      "author.last" : 1,
      "author.middle": {
        $cond: {
          if: { $eq: [ "", "$author.middle" ] },
          then: "$$REMOVE",
          else: "$author.middle"
        }
      }
    }
  }
] )

```

The aggregation operation results in the following output:

```

{ "_id" : 1, "title" : "abc123", "author" : { "last" : "zzz", "first" : "aaa" } }
{ "_id" : 2, "title" : "Baked Goods", "author" : { "last" : "xyz", "first" : "ak" } }
{ "_id" : 3, "title" : "Ice Cream Cakes", "author" : { "last" : "xyz", "first" : "ak" } }

```

You can use either the `$addFields` or `$project` stage to remove document fields. The best approach depends on your pipeline and how much of the original document you want to retain.

To see an example using `$$REMOVE` in an `$addFields` stage, see [Remove Fields](#).

Include Specific Fields from Embedded Documents

Consider a `bookmarks` collection with the following documents:

```

{ _id: 1, user: "1234", stop: { title: "book1", author: "xyz", page: 32 } }
{ _id: 2, user: "7890", stop: [ { title: "book2", author: "abc", page: 5 }, { title: "book3", author: "def", page: 10 } ] }

```

To include only the `title` field in the embedded document in the `stop` field, you can use the dot notation:

```
db.bookmarks.aggregate( [ { $project: { "stop.title": 1 } } ] )
```

Or, you can nest the inclusion specification in a document:

```
db.bookmarks.aggregate( [ { $project: { stop: { title: 1 } } } ] )
```

Both specifications result in the following documents:

```
{ "_id" : 1, "stop" : { "title" : "book1" } }  
{ "_id" : 2, "stop" : [ { "title" : "book2" }, { "title" : "book3" } ] }
```

Include Computed Fields

Consider a `books` collection with the following document:

```
{  
  "_id" : 1,  
  "title": "abc123",  
  "isbn": "0001122223334",  
  "author": { last: "zzz", first: "aaa" },  
  "copies": 5  
}
```

The following `$project` stage adds the new fields `isbn`, `lastName`, and `copiesSold`:

```

db.books.aggregate(
  [
    {
      $project: {
        title: 1,
        isbn: {
          prefix: { $substr: [ "$isbn", 0, 3 ] },
          group: { $substr: [ "$isbn", 3, 2 ] },
          publisher: { $substr: [ "$isbn", 5, 4 ] },
          title: { $substr: [ "$isbn", 9, 3 ] },
          checkDigit: { $substr: [ "$isbn", 12, 1 ] }
        },
        lastName: "$author.last",
        copiesSold: "$copies"
      }
    }
  ]
)

```

The operation results in the following document:

```

{
  "_id" : 1,
  "title" : "abc123",
  "isbn" : {
    "prefix" : "000",
    "group" : "11",
    "publisher" : "2222",
    "title" : "333",
    "checkDigit" : "4"
  },
  "lastName" : "zzz",
  "copiesSold" : 5
}

```

Project New Array Fields

For example, if a collection includes the following document:

```
{ "_id" : ObjectId("55ad167f320c6be244eb3b95"), "x" : 1, "y" : 1 }
```

The following operation projects the fields `x` and `y` as elements in a new field `myArray` :

```
db.collection.aggregate( [ { $project: { myArray: [ "$x", "$y" ] } } ] )
```

The operation returns the following document:

```
{ "_id" : ObjectId("55ad167f320c6be244eb3b95"), "myArray" : [ 1, 1 ] }
```

If the array specification includes fields that are non-existent in a document, the operation substitutes `null` as the value for that field.

For example, given the same document as above, the following operation projects the fields `x`, `y`, and a non-existing field `$someField` as elements in a new field `myArray` :

```
db.collection.aggregate( [ { $project: { myArray: [ "$x", "$y", "$someField" ] } } ] )
```

The operation returns the following document:

```
{ "_id" : ObjectId("55ad167f320c6be244eb3b95"), "myArray" : [ 1, 1, null ] }
```

Array Indexes are Unsupported

You cannot use an array index with the `$project` stage. This section shows an example.

Create the following `pizzas` collection:

```
db.pizzas.insert( [
  { _id: 0, name: [ 'Pepperoni' ] },
] )
```

The following example returns the pizza:

```
db.pizzas.aggregate( [
  { $project: { x: '$name', _id: 0 } },
] )
```

The pizza is returned in the example output:

```
[ { x: [ 'Pepperoni' ] } ]
```

The following example uses an array index (`$name.0`) to attempt to return the pizza:

```
db.pizzas.aggregate( [
  { $project: { x: '$name.0', _id: 0 } },
] )
```

The pizza is not returned in the example output:

```
[ { x: [] } ]
```

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB .NET/C# Driver documentation](#).

The following `Movie` and `ImdbData` classes model the documents in the `sample_mflix.movies` collection:

```

public class Movie
{
    public ObjectId Id { get; set; }

    public string Title { get; set; }

    public List<string> Genres { get; set; }

    public List<string> Directors { get; set; }

    public List<string> Writers { get; set; }

    public string Type { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public List<string> Cast { get; set; }
}

```

```

public class ImdbData
{
    public string Id { get; set; }

    public int Votes { get; set; }

    public float Rating { get; set; }
}

```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```

var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);

```

To use the MongoDB .NET/C# driver to add a `$project` stage to an aggregation pipeline, call the `Project()` method on a `PipelineDefinition` object and pass a

`ProjectionDefinitionBuilder<TDocument>` object. `TDocument` is the class that represents the documents in your collection.

The following sections show the different ways that you can customize the output documents of the `$project` stage.

Include Specific Fields in Output Documents

To include specific fields when using the .NET/C# driver, call the `Include()` method on the projection builder. You can chain `Include()` calls to include multiple fields.

The following code example produces a document that includes only the `_id`, `plot`, and `title` fields:

```
var pipeline = new EmptyPipelineDefinition<Movie> ()
    .Project(
        Builders<Movie>.Projection
            .Include(m => m.Title)
            .Include(m => m.Plot)
    );

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();
```

The pipeline returns the following document:

```
{
  "_id" : { "$oid" : "573a1390f29313caabcd42e8" },
  "plot" : "A group of bandits stage a brazen train hold-up, only to find a
           determined posse hot on their heels.",
  "title" : "The Great Train Robbery"
}
```

Exclude Fields from Output Documents

To exclude a field from the result documents when using the .NET/C# driver, call the `Exclude()` method on the projection builder. You can chain `Exclude()` calls to exclude multiple fields.

The following code example produces a document that excludes the `Type` field:

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Project(
        Builders<Movie>.Projection
            .Exclude(m => m.Type)
    );

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();
```

By default, result documents always include the `_id` field. The following code example produces a document that excludes the `_id` field but includes the `plot` and `title` fields:

```
var pipeline = new EmptyPipelineDefinition<Movie> ()
    .Project(
        Builders<Movie>.Projection
            .Exclude(m => m.Id)
            .Include(m => m.Title)
            .Include(m => m.Plot)
    );

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();
```

The pipeline results in the following document:

```
{
  "plot" : "A group of bandits stage a brazen train hold-up, only to find a
           determined posse hot on their heels.",
  "title" : "The Great Train Robbery"
}
```

Exclude Fields from Embedded Documents

To exclude a field in an embedded document when using the .NET/C# driver, call the `Exclude()` method on the projection builder and pass the path to the corresponding class property. You can chain `Exclude()` calls to exclude multiple fields.

The following code example produces a document that excludes the `imdb.id` and `type` fields:

```
var pipeline = new EmptyPipelineDefinition<Movie> ()
    .Project(
        Builders<Movie>.Projection
            .Exclude("Imdb.id")
            .Exclude(m => m.Type)
    );

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();
```

The pipeline results in the following output:

```
{
  "plot" : "A group of bandits stage a brazen train hold-up, only to find a
           determined posse hot on their heels.",
  "title" : "The Great Train Robbery",
  ...
  "imdb" : { "rating" : 7.4000000000000004, "votes" : 9847 }
}
```

To project an ID field in an embedded document, specify the field name as a string, not a lambda expression.

Conditionally Exclude Fields

You can use the variable `REMOVE` in aggregation expressions to conditionally suppress a field:

```

var stage = new BsonDocument
{
    { "title", 1 },
    { "imdb.id", 1 },
    { "imdb.rating", 1 },
    {
        "imdb.votes", new BsonDocument("$cond", new BsonDocument
        {
            { "if", new BsonDocument("$eq", new BsonArray { "", "$imdb.votes" }},
            { "then", "$$REMOVE" },
            { "else", "$imdb.votes" }
        })
    }
};

var pipeline = new EmptyPipelineDefinition<Movie>()
    .Project(stage)
    .Sample(1);

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();

```

The preceding example uses `BsonDocument` objects because the .NET/C# driver doesn't provide a builder to conditionally exclude fields. Other MongoDB language drivers might support this feature. See the MongoDB driver documentation for more information.

If the sampled document contains the `imdb.votes` field, the pipeline returns a document similar to the following:

```

{
  "_id" : { "$oid" : "573a1390f29313caabcd42e8" },
  "title" : "The Great Train Robbery",
  "imdb" : { "rating" : 7.4000000000000004, "id" : 439, "votes" : 9847 }
}

```

If the document doesn't contain the `imdb.votes` field, the pipeline returns a document similar to the following:

```
{
  "_id" : { "$oid" : "573a1398f29313caabce94a3" },
  "title" : "This Is Spinal Tap",
  "imdb" : { "rating" : 8.0, "id" : 88258 }
}
```

Include Computed Fields

To include computed fields in the result documents when using the .NET/C# driver, call the `Expression()` method on the projection builder and pass an expression that includes the computed fields. For added type safety, you can define a model class for the result documents, like the following `ProjectedMovie` class:

```
public class ProjectedMovie
{
    public ObjectId Id { get; set; }

    public string Title { get; set; }

    public string LeadActor { get; set; }

    public List<string> Crew { get; set; }
}
```

The following code example produces a document that includes multiple computed fields:

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Project(
        Builders<Movie>
            .Projection
                .Expression(m => new ProjectedMovie
                    {
                        Id = m.Id,
                        Title = m.Title,
                        LeadActor = m.Cast[0],
                    })
    );

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();
```

The pipeline results in the following document:

```
{
  "_id" : { "$oid" : "573a1390f29313caabcd42e8" },
  "title" : "The Great Train Robbery",
  "leadActor" : "A.C. Abadie",
  ...
}
```

Project New Array Fields

To project new array fields in the result documents when using the .NET/C# driver, call the `Expression()` method on the projection builder and pass an expression that includes the new array fields. For added type safety, you can define a model class for the result documents, like the following `ProjectedMovie` class:

```
public class ProjectedMovie
{
    public ObjectId Id { get; set; }

    public string Title { get; set; }

    public string LeadActor { get; set; }

    public List<string> Crew { get; set; }
}
```

The following code example produces documents that include a new array field, `crew`, which contains values from the `directors` and `writers` fields:

```

var pipeline = new EmptyPipelineDefinition<Movie> ()
    .Project(
        Builders<Movie>
            .Projection
            .Expression(m => new ProjectedMovie
                {
                    Id = m.Id,
                    Title = m.Title,
                    LeadActor = m.Cast[0],
                    Crew = m.Directors.Concat(m.Writers).ToList()
                }
            )
    )
    .Sample(1);

var result = movieCollection.Aggregate(pipeline).FirstOrDefault();

```

The pipeline returns a document similar to the following:

```

{
  "_id" : { "$oid" : "573a1395f29313caabce2297" },
  "title" : "The Chalk Garden",
  "leadActor" : "Deborah Kerr",
  "crew" : ["Ronald Neame", "John Michael Hayes (screenplay)", "Enid Bagnold (f
}

```

If the array specification includes fields that are non-existent in a document, the pipeline substitutes `null` as the value for that field. For example, the following code example projects the fields `directors`, `writers`, and a non-existent field, `makeupArtists`, as elements in a new field named `crew`:

```

var stage = new BsonDocument
{
    { "crew", new BsonArray { "$directors", "$writers", "$makeupArtists" } }
};

var pipeline = new EmptyPipelineDefinition<Movie>()
    .Project(stage)
    .Sample(1);
var result = movieCollection.Aggregate(pipeline).FirstOrDefault();

```

The pipeline returns a document similar to the following:

```

{
  "_id" : { "$oid" : "573a1399f29313caabced0d9" },
  "crew" : [ ["Bill Kroyer"], ["Jim Cox (screenplay)", "Diana Young (original story)"] ]
}

```

The preceding example uses `BsonDocument` objects because the .NET/C# driver raises a compile-time error if you try to use builders to add a missing field to an array. Other MongoDB language drivers might support this feature. See the MongoDB driver documentation for more information.

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started](#) in the MongoDB Node.js driver documentation.

To use the MongoDB Node.js driver to add a `$project` stage to an aggregation pipeline, use the `$project` operator in a pipeline object.

The following sections show how to customize the output documents of the `$project` stage.

Include Specific Fields in Output Documents

To include specific fields, set the value of the field to `1` in the `$project` stage.

The following example returns documents that include only the `_id`, `plot`, and `title` fields:

```
const pipeline = [
  {
    $project: {
      title: 1,
      plot: 1
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  _id: new ObjectId('573a1390f29313caabcd42e8'),
  plot: 'A group of bandits stage a brazen train hold-up, only to find a
        determined posse hot on their heels.',
  title: 'The Great Train Robbery'
}
```

Exclude Fields from Output Documents

To exclude specific fields, set the value of the field to `0` in the `$project` stage.

The following example returns documents that exclude the `type` field:

```
const pipeline = [
  {
    $project: {
      type: 0
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

By default, result documents always include the `_id` field. The following example returns documents that exclude the `_id` field but include the `plot` and `title` fields:

```
const pipeline = [
  {
    $project: {
      _id: 0,
      title: 1,
      plot: 1
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  plot: 'A group of bandits stage a brazen train hold-up, only to find a
        determined posse hot on their heels.',
  title: 'The Great Train Robbery'
}
```

Exclude Fields from Embedded Documents

To exclude a field in an embedded document, set value of the field path to `0` in the `$project` stage.

To project a field in an embedded document, specify the field path as a string.

The following example returns documents that exclude the `imdb.id` and `type` fields:

```
const pipeline = [
  {
    $project: {
      "imdb.id": 0,
      type: 0
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  plot: 'A group of bandits stage a brazen train hold-up, only to find a
        determined posse hot on their heels.',
  title: 'The Great Train Robbery',
  imdb: { rating: 7.4000000000000004, votes: 9847 }
}
```

Conditionally Exclude Fields

To conditionally exclude a field, assign conditional logic that includes the variable `REMOVE` to the field name.

```

const pipeline = [
  {
    $project: {
      title: 1,
      "imdb.id": 1,
      "imdb.rating": 1,
      "imdb.votes": {
        $cond: {
          if: { $eq: ["$imdb.votes", ""] },
          then: "$REMOVE",
          else: "$imdb.votes"
        }
      }
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;

```

If a document contains the `imdb.votes` field, the pipeline returns documents that resemble the following example output:

```

{
  _id: new ObjectId('573a1390f29313caabcd42e8'),
  title: 'The Great Train Robbery',
  imdb: { rating : 7.4000000000000004, id: 439, votes: 9847 }
}

```

If a document doesn't contain the `imdb.votes` field, the pipeline returns documents that resemble the following example output:

```

{
  _id: new ObjectId('573a1390f29313caabcd42e8'),
  title: 'This Is Spinal Tap',
  imdb: { rating: 8.0, id: 88258 }
}

```

Include Computed Fields

To include computed fields in the result documents, assign an expression to the field that stores the results.

The following example projects the `_id` and `title` fields into new fields of the same name and computes the new field `leadActor`. The example then returns documents that include these fields:

```
const pipeline = [
  {
    $project: {
      _id: "$_id",
      title: "$title",
      leadActor: { $arrayElemAt: ["$cast", 0] }
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  _id: new ObjectId('573a1390f29313caabcd42e8'),
  title: 'The Great Train Robbery',
  leadActor: 'A.C. Abadie'
}
```

Project New Array Fields

To project new array fields in the result documents, assign an expression that computes the new array field to the field that stores the array.

The following example returns documents that include a new array field, `crew`, which combines values from the `directors` and `writers` fields:

```
const pipeline = [
  {
    $project: {
      _id: "$_id",
      title: "$title",
      leadActor: { $arrayElemAt: ["$cast", 0] },
      crew: { $concatArrays: ["$directors", "$writers"] }
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  _id: new ObjectId('573a1395f29313caabce2297'),
  title: 'The Chalk Garden',
  leadActor: 'Deborah Kerr',
  crew: ['Ronald Neame', 'John Michael Hayes (screenplay)', 'Enid Bagnold (from
}
```

If the array specification includes fields that aren't in a document, the pipeline substitutes `null` as the value for that field. For example, the following example projects the fields `directors`, `writers`, and a non-existent field, `makeupArtists`, as elements in a new field named `crew`:

```
const pipeline = [
  {
    $project: {
      crew: ["$directors", "$writers", "$makeupArtists"]
    }
  }
];

const cursor = collection.aggregate(pipeline);
return cursor;
```

The output documents resemble the following example:

```
{
  _id: new ObjectId('573a1399f29313caabced0d9'),
  crew: [['Bill Kroyer'], ['Jim Cox (screenplay)', 'Diana Young (original storie
}
```

Learn More

For more information and detailed explanations of common aggregation tasks, see the [Complete Aggregation Pipeline Tutorials](#).

To learn more about the methods on this page, see the following guides:

- [\\$eq](#)
- [\\$cond](#)
- [\\$arrayElemAt](#)
- [\\$concatArrays](#)
- [REMOVE](#)